Agentic Engineering: Five Papers

A pragmatic reader's guide — 1 June 2026

From the arXiv daily agentic-engineering digest · 1 June 2026

1 June 2026

Chapter 0: The Harness Is the Product Now

Read these five papers back to back and a single argument assembles itself, even though no two author teams set out to make it. The model is no longer the interesting variable. The system wrapped around it — the memory it reloads, the skills it loads, the context it builds, the trajectories it learns from — is where capability, risk, and the next round of gains all now live. Each paper takes a different slice of that system and arrives, independently, at a few of the same convictions. Those convictions are worth naming up front, because once you see them you'll spot them recurring in every chapter.

Capability and attack surface are the same machinery

The opening paper's thesis is that long-horizon agent performance is gated by the harness, not the weights — token budget, tool calls, and orchestration explained 95% of the performance variance in one production system. The second paper delivers the corollary nobody wants: the very memory stores, persistent files, and skill manifests that make an agent capable are exactly what an attacker wants to own. A poisoned `CLAUDE.md` and a useful `CLAUDE.md` are the same file. When you read

Chapter 1's case for investing in memory and skill infrastructure, keep Chapter 2's multi-step trojan in your peripheral vision. They are describing the same surface from opposite sides — one as the source of leverage, the other as the thing you now have to defend.

Stored artifacts are hypotheses, not facts

The sharpest shared instinct across these papers is a refusal to trust anything just because it was written down and ranked highly. Chapter 1 argues that retrieved memory is a hypothesis until re-verified against the live environment — trust is a runtime decision, not a property baked into the stored item. Chapter 2 enforces the same idea defensively: untrusted content carries a provenance label across every step, and may be used as data but never silently become future instruction. Chapter 3 assumes a freshly generated skill is *wrong on arrival*, and builds correction, rollback, and versioning into the artifact so it can be repaired rather than trusted. Three different problems — stale memory, poisoned files, imperfect distillation — converge on one discipline: attach provenance and confidence to everything, verify at the moment of use, and keep a way to roll back. If you carry one habit out of this bundle, make it that one.

Supervise the process, not just the endpoint

A second convergence runs through the back half. The single success number is a liar — it hides how the answer was reached, and rewards the agent that guessed right for the wrong reasons. Chapter 1 makes this an evaluation argument: report process metrics (tokens, retries, what was retrieved, what was verified) alongside the endpoint score, and weld a post-condition check to every skill. Chapters 4 and 5 make it a *training* argument. GrepSeek manufactures verified, causally-honest search trajectories and discards any whose reasoning peeked at the answer. LongTraceRL hands out entity-level rubric rewards for surfacing the right facts along the chain — but only to answers that were already correct, so the process reward becomes a tiebreaker rather than an invitation to game. Same conviction, three altitudes: in evaluation, in skill execution, and in the reward function, the intermediate steps are where the truth is.

Trajectories are the raw material

Notice what the last three papers are all quietly doing: mining traces. COLLEAGUE.SKILL distills scattered *human* work traces into a loadable skill. GrepSeek splits an answer-aware Tutor and an answer-blind Planner to manufacture verified *search* trajectories nobody had to hand-label. LongTraceRL lets a real search agent answer a question, then harvests the documents it opened-but-didn't-cite as hard negatives and the gold entities it crossed as process supervision. The trajectory — human or agentic, successful or discarded — has become the substrate you build training data and skills out of. If your agents run in production, you are already generating this

material and almost certainly throwing it away. Several of these papers are, in effect, instructions for mining your own logs.

Smaller models, better systems

Underneath all of it sits the bundle's quiet rebuke to scale-maximalism. A 9B model with a learned grep habit beats RL-tuned agents wired to strong dense retrievers. A 4B model picks up real reasoning gains from harder distractors and a gated reward. The harness paper makes the thesis explicit; the others keep proving it by example. The leverage has moved from the weights to the system design — which is good news, because system design is something a small team can actually iterate on.

What to carry into the chapters

Read Chapter 1 as the map and the other four as expeditions into specific territories it names: memory and trust (Chapters 1, 2, 3), skills as first-class versioned artifacts (Chapters 2, 3), and learned search-and-reason behavior built from trajectories (Chapters 4, 5). Watch for the recurring moves — provenance labels, verify-at-use, process-level supervision, mine-the-trajectory — and notice how a defense paper, a distillation paper, and two RL papers keep reaching for the same handful of tools. That repetition is the signal. The field is converging on a way of building agents, and these five papers are five angles on it.

Chapter 1: Scaling the Harness, Not the Model

Paper: From Model Scaling to System Scaling: Scaling the Harness in Agentic AI · arXiv 2605.26112 · **Code:** github.com/SafeRL-Lab/cheetahclaws

Here is a number worth sitting with. In Anthropic's own multi-agent research system, token usage alone explained 80% of the variance in performance. Add tool-call count and model choice, and you reach 95%. Notice what is doing the heavy lifting there: not how smart the model is, but how the system around it spends its budget. That, in one statistic, is the argument of this paper. The next wave of gains in agentic AI is going to come less from a better brain and more from a better body around the brain.

The score you report is a lie about the model

Shangding Gu, a Berkeley researcher, opens with a diagnosis the field has been quietly avoiding. We still grade agents the way we grade models: did it pass the benchmark, what was the final accuracy. But once a model is wired into a terminal, a repo, a browser, and a memory store, its behavior stops being a property of the model. It becomes a property of a *system* — how context gets built, how memory gets retrieved, how tools get invoked, how actions get verified.

The evidence is embarrassing for the old framing. Redesign the agent-computer interface alone — same model underneath — and SWE-bench accuracy jumps substantially. A field-wide audit found many "model" results stop being Pareto-optimal once you control for prompting strategy and cost. The blunt conclusion: what we keep calling a model score is really a **model-plus-harness score**, and we have been crediting the wrong half.

Gu's name for the half we ignore is the **harness** — the structured execution layer wrapping the model. Tool interface, control loop, context constructor, memory store, skill router, and a verification-and-governance layer. The paper's claim is that once a model crosses some capability threshold, long-horizon performance is gated by the harness, not the weights. He even writes it as a back-of-envelope decomposition: performance equals some function of reasoning (R), memory (M), context (C), skill routing (S), orchestration (O), and governance (G). Model scaling buys you better R. Everything else is system scaling, and it has been treated as plumbing.

Three places the plumbing leaks

Gu zeroes in on three bottlenecks, and the useful move is that he names each one's failure mode in a way you can actually go hunting for.

Context governance is the first, and the reframe is sharp: the hard problem of context is not capacity, it is governance. A bigger window does not help if the model attends to padding instead of evidence — attention dilutes over long inputs, and models famously favor the start and end of a context over the middle. The failure mode he names is *exposure without access*: the right token is technically present but functionally invisible. The fix is to treat each turn's context as the output of a selection policy with a token budget and provenance tracking, not as a buffer you keep appending to.

Trustworthy memory is the second, and it is the one most teams will recognize in their own logs. The failure is *stale-but-confident*. A note that said "the data loader lives in utils/loader.py" stays highly ranked by semantic search long after a refactor made it false — and acting on it is now actively destructive. The insight: trust should be a runtime decision made at retrieval time, not a property baked into the stored item. Retrieved memory is a hypothesis until re-verified against the live environment. Gu points at Claude Code's hybrid design as the working example — `CLAUDE.md` carries durable priors while glob/grep re-check the actual repo on demand, so the agent keeps accumulated knowledge without trusting a stale index.

Dynamic skill routing is the third, and its failure mirrors the second: *confident-but-unchecked*. As you add specialized subagents, the bottleneck shifts from "we lack a capability" to "a subagent returned plausible output and nobody validated it." Gu's analogy is the OS scheduler — raw skill capacity exists, but useful work depends on routing it to the right pathway at the right time, with a post-condition check welded to every skill. His sharpest line here: scaling skill quality without scaling verification just buys you faster, less reliable progress.

Same model, three different animals

To make this concrete instead of philosophical, the paper releases **CheetahClaws**, a Python-native reference harness, and sets it beside Claude Code and OpenClaw. The point is not a leaderboard. It is that comparable models, projected onto different harnesses, behave like genuinely different agents. All three consolidate sessions into persistent memory, but they represent *trust* differently — CheetahClaws stores per-entry confidence and recency as first-class fields used directly in retrieval ranking, while the others infer trust implicitly from access patterns. Turn governance off, or run the orchestration loop one-shot, and the same R, C, and S start acting like a noticeably worse agent. The harness is the variable.

Where the leverage actually is

The practical payload here is an audit checklist, and you do not need to run Gu's experiment to use it. Stop reporting a single success number; report process metrics alongside it — tokens, tool calls, retries, failed edits, what was retrieved, what was verified. Two agents can both pass and differ wildly in cost and risk, and the endpoint score hides exactly the things that determine whether the thing is deployable.

Then walk the three failure modes through your own stack. For context: is anything appended to the window without a relevance gate and a budget? For memory: when you retrieve a stored fact, do you re-verify it against the live environment, or do you act on it because it ranked high? For skills and subagents: does every subagent's output hit a post-condition check, or do you trust it because it sounded right? Borrow CheetahClaws' move directly — attach a confidence score and a last-verified timestamp to memory entries, and let a staleness penalty fall out of that for free. And steal the longitudinal lens: stop resetting your agent between tasks in evaluation. Real agents accumulate state, and the same accumulation that makes them improve is what lets memory quietly rot. A one-shot test cannot tell you whether your agent's memory is getting more useful or more dangerous over a week of use.

If this chapter's argument is that the harness is where capability now lives, the next one delivers the uncomfortable corollary: the harness is also where the attack surface lives. The very memory stores, persistent files, and tool permissions that make an agent capable are the same machinery an adversary wants to own. The following chapter follows a single prompt injection as it escalates from a one-off nuisance into persistent backdoor control of an agent's workspace — and lays out a Detect-Attribute-Sanitize defense for clawing that workspace back.

Chapter 2: The Backdoor That Looks Like a Note

Paper: From Prompt Injection to Persistent Control: Defending Agentic Harness Against Trojan Backdoors · arXiv 2605.31042 · **Code:** github.com/RUC-NLPIR/ClawTrojan

Here is the uncomfortable finding, up front: the strongest models on the market have basically solved single-shot prompt injection, and that has quietly created a more dangerous blind spot. When the team behind this paper ran the old standby benchmarks (AgentDojo, InjecAgent) against GPT-5.4, the attack success rate was zero. The model reads the "ignore your instructions and exfiltrate the secrets" payload, recognizes it as a contaminant, and refuses. Defense solved, right?

Then they built **ClawTrojan**, a benchmark where no single step ever contains that obvious payload, and the same model failed 95.5% of the time. That gap is the whole story.

What actually changed

The shift is from one malicious prompt to a **multi-step trojan**: an attack spread across files, memory, and tool returns inside a local agent workspace, where each piece looks like ordinary work. A project note says release reports need a short diagnostic block. A config file later defines that block as "text copied from `private_notes.txt`." Neither line is alarming. But when you ask for the release report, the agent dutifully copies private text into a shared document. The harm is real; it just never lived in a single step a detector could flag.

The authors group these into a few recognizable patterns. **Memory poisoning** plants a rule in persistent memory that gets reloaded next session. **Trust laundering** lets the agent do a few legitimate tasks first, so an untrusted "approved exception" gets written into a runbook as if it were verified policy. **Skill poisoning** hides capability in a skill's README or metadata that contradicts its declared manifest. The common thread: untrusted *data* becomes persistent *control content* — instructions, policies, and action targets the harness trusts in future runs.

This is why existing defenses miss it. ClawKeeper, StruQ, MELON, PromptShield — all inspect the current prompt or the immediate action. They can block a visibly dangerous send or delete, but they never connect that final action back to the innocuous file write three steps earlier that planted the backdoor. Against ClawTrojan they barely move the needle (most still sit above 88% ASR). Even CaMeL, with real data-flow tracking, only gets down to 74%.

Why this should worry anyone running agents in production

If your agent has a workspace — memory it reloads, `AGENTS.md` or `CLAUDE.md` files it reads, skills it loads, project docs it trusts — you have this attack surface, whether or not anyone has aimed it at you yet. The benchmark is OpenClaw-style, but the authors are explicit that the structure generalizes: nearly every local harness shares the same parts (conversation history, memory, project files), and that is all the attack needs.

The deeper reframe is the one worth internalizing. The security question is no longer "is this input malicious?" It is "has untrusted content become future instruction or policy?" An attack that fails to leak anything *today* can still succeed if it gets a rule committed to the workspace, because that rule sits there waiting for a later session to make it actionable. The paper names the **last intervention point** — the step after which the attack can no longer be stopped — and it is almost always an early, boring file write, not the dramatic final action everyone is watching.

The defense, and what to borrow from it

Their answer is **DASGuard** — Detect, Attribute, Sanitize — and it cuts ASR to 15.8%. The mechanics are worth understanding because they tell you where to put your own guardrails. DASGuard sits on the harness boundary and, for every proposed operation, asks two questions: does this payload contain a control-bearing span (a directive, a memory rule, a policy shift), and is it being written into something *sensitive* — memory, policy files, skill instructions, action targets? It labels every piece of content by provenance (trusted = user and system text; untrusted = external tool returns, downloaded files, anything overlapping a prior finding), and it carries those labels *across steps*. The governing rule is one sentence: untrusted data may be used as data, but it must not become future instructions or high-risk action targets unless the user clearly authorizes it.

Two implementation details earned their keep in the ablation, and both are easy to lift conceptually. First, **inspect writes, not just actions** — DASGuard applies file edits to a shadow copy and scans only the changed spans before committing. Second, **provenance is everything**: stripping the source labels was catastrophic, sending ASR back up to 92.7%. Cross-step memory and embedding matching helped, but provenance was the load-bearing wall.

So, concretely, if you are hardening an agent: figure out what *persistent control content* means in your harness — which files, when reloaded, actually steer behavior — and gate writes into those specifically, separately from gating the splashy external actions. Tag content by where it came from and keep that tag alive as the content gets copied and combined. Treat a skill's prose that quietly implies messaging or credential access as a control-flow finding, not as documentation. And note the honest cost: DASGuard's false-positive rate climbs to 13%, so this isn't free — it's a "send borderline writes to the user for review" trade, which is tolerable precisely because the attack surface shrinks so much.

The thread running through all of this is that an agent's memory and skills are now a place attackers write to, not just a place you read from. Which raises an obvious and slightly unnerving question for the next chapter: if a poisoned skill is this dangerous, how much should we trust the skills we *manufacture* ourselves? COLLEAGUE.SKILL takes the optimistic side of that coin — automatically distilling reusable agent skills from expert human traces, capturing capabilities, mental models, behavioral constraints, and even correction history at scale. Read alongside this chapter, it sharpens the stakes: skills are becoming both the thing we mass-produce and the thing we can no longer afford to trust blindly.

Chapter 3: The Coworker You Can Read, Roll Back, and Delete

Paper: COLLEAGUE.SKILL: Automated AI Skill Generation via Expert Knowledge Distillation · arXiv 2605.31264 · **Code:** github.com/titanwings/colleague-skill

When a senior engineer leaves a team, the postmortem instincts, the review reflexes, the “we don’t ship without rate limiting” muscle memory all walk out the door with them. None of it was ever written down as a clean instruction manual. It lived in code-review comments, incident notes, and Slack decisions. COLLEAGUE.SKILL is a system that scoops up exactly those messy traces and renders them into a portable agent skill you can open, correct, and delete. The pitch is deliberately modest: it is not trying to clone the person. It is trying to turn scattered evidence of how they work into a versioned folder an agent can load on demand.

What it actually builds

Give the system some materials about a person or role — chat logs, work docs, email, screenshots, subtitles, a few lines of description — and it runs them through a distillation pipeline. Collectors normalize the raw stuff into a local knowledge directory. Analyzers pull out durable capability, mental models, and interaction style. A writer then emits a **versioned skill package** built on two coordinated tracks.

The first is a **capability track** (`work.md`): responsibilities, workflows, technical standards, review criteria, decision heuristics, lessons learned. The second is a **behavior track** (`persona.md`): communication style, interaction rules, and — importantly — a running correction log. The split matters. The paper argues that most persona systems fail because they blur three different things into one prompt: factual knowledge, procedural judgment, and surface tone. By keeping capability and behavior in separate, inspectable files, you can invoke the full skill, the capability-only version, or the style-only version independently. When you want a coworker’s review judgment but not their voice, you load just the work track.

The package follows the Agent Skills standard: a required `SKILL.md` entrypoint with frontmatter and `user-invocable: true` , plus split sub-skill entrypoints, a `manifest.json` for installers and slash commands, and a `meta.json` carrying schema version, provenance, lifecycle version, and correction count. It installs across hosts like Claude Code, OpenClaw, Codex, and Hermes. Three presets reuse the identical workflow under different evidence and consent rules: **colleague** (enterprise work traces), celebrity/public-figure (public first-person sources with citation discipline), and relationship (private traces under strong local-control and deletion defaults).

Why this is a different bet than the usual skill-synthesis paper

Most recent skill-library work distills *agent* trajectories — successful and failed runs of the machine itself — into reusable strategies. COLLEAGUE.SKILL distills *human* traces instead, and it treats the whole thing as a workflow over files rather than a one-shot generation. That reframing is the real contribution, and it's worth borrowing even if you never touch this codebase.

The lifecycle is the point. A generated skill is assumed to be wrong on arrival. So the system makes creation, inspection, invocation, correction, rollback, deletion, host installation, and optional distribution into first-class operations. Feedback arrives in plain language — "he wouldn't say that," "she'd push back here." If it concerns expert work, the handler produces a Markdown patch: matching level-2 headings replace their section, unmatched ones get appended. If it concerns behavior, it writes a normalized `{scene, wrong, correct}` record. Either way the writer archives the current version, applies the change, bumps the lifecycle version, and regenerates every derived artifact. You can list archived versions, roll back, and prune old ones.

This is the move that elevates the paper above prompt engineering: **correction history becomes a durable artifact**, not a vanished chat turn. The honest part is that the authors make artifact-level claims only. They do *not* assert the generated skill catches the same bugs the source expert would, and they explicitly flag that corrections can encode editor bias or make contested traces look more settled than they are. The deployment numbers they cite — roughly 18.5k GitHub stars, a 215-skill gallery — are offered as evidence of distribution surface, not adoption quality. That restraint is refreshing in a space full of fidelity hype.

What to steal for your own skill library

If you maintain skills for agents, several pieces transfer cleanly. First, split capability from persona in storage even if you usually invoke them together — it gives you a knob for "expertise without voice," which is exactly what you want when style transfer would be inappropriate or creepy. Second, capture more than instructions from your traces: pull out decision heuristics and escalation thresholds (an engineer's "check auth, input validation, rate limiting, response schema, and data exposure *before* lower-priority issues"), not just procedures. Third, and most actionable: make correction a logged, structured patch against versioned state rather than an edit-in-place. A `{scene, wrong, correct}` record plus archive-then-regenerate gives you an auditable history and a rollback point for free, and it turns "the skill drifted" into a diff you can read. Fourth, let metadata carry the governance — source boundaries, provenance, deletion paths in a manifest — so withholding or sharing a skill is a property of the file, not tribal knowledge. Even outside person-grounded skills, treating any generated capability as a correctable, versioned package beats treating it as frozen prompt text.

The unglamorous lesson is that productization *is* the research here. Installers, manifests, rollback state, and deletion paths are what let a distilled skill be audited, repaired, or withheld — the conditions under which a self-evolving skill library can actually be trusted to evolve.

COLLEAGUE.SKILL manufactures knowledge from traces: it takes evidence of how a person works and bottles it into a loadable artifact. The next chapter inverts the question. Instead of handing an agent a pre-packaged skill or a fixed retriever, GrepSeek trains the *search behavior itself* — teaching an agent to root through a raw text corpus with shell commands like `grep` and file reads, first by learning from verified trajectories produced by a Tutor and Planner, then sharpening that behavior with GRPO reinforcement learning. Where this chapter asked how to package what an expert knows, the next asks how an agent should go find what it doesn't.

Chapter 4: Giving the Agent a Terminal Instead of a Vector Database

Paper: GrepSeek: Training Search Agents for Direct Corpus Interaction · arXiv 2605.29307 · **Code:** github.com/alirezasalemi7/grepseek

Here is the practical bet GrepSeek makes: most of the machinery we wrap around retrieval-augmented agents — the embedding model, the vector index, the offline indexing job, the 200GB of precomputed representations — might be optional. Instead of giving your agent a `search(query)` API backed by a retriever, you give it a shell prompt and let it run `rg`, `grep`, `head`, and `sed` directly against the raw corpus. The corpus *is* the environment. The agent finds evidence the way a senior engineer finds a bug in an unfamiliar repo: by grepping, reading a few lines, narrowing the pattern, and grepping again.

The headline result is that this works, and on the hardest questions it works *better*. On a 21-million-passage Wikipedia corpus, a compact 9B model trained this way beats index-based RAG, untrained agentic frameworks, and even RL-tuned agents wired to strong dense retrievers — winning on four of seven QA benchmarks and taking the best overall token-level F1. The gains concentrate exactly where you'd hope: multi-hop reasoning, where dense retrievers blur distinct-but-similar entities into the same embedding neighborhood. And it does this with a 14GB memory footprint (just the raw text) versus 70GB for an E5 index or 221GB for a 4B-embedding index, with offline indexing cost dropping from dozens of A100-hours to about a minute of setup.

Why You Have to Train This, Not Just Prompt It

The tempting shortcut — and what a couple of contemporary papers did — is to hand the whole thing to a big proprietary model like Claude and let it orchestrate shell search at inference time. It works, but it's slow (sometimes an hour per query) and expensive. GrepSeek's contribution is making **direct corpus interaction** a *learned* capability of a small model rather than an emergent behavior of a giant one.

That turns out to be hard. Naively pointing reinforcement learning at a base model and rewarding correct answers produces degenerate behavior: overly broad commands, agents that `cat` half the corpus into their context window, out-of-memory crashes even on 1TB machines. The search behavior you want — surgical, exact-match, iteratively filtered — doesn't fall out of RL on its own. You have to teach the primitives first.

The Tutor-and-Planner Trick for Manufacturing Good Trajectories

The cleverest piece here is how GrepSeek generates its cold-start training data, and it's the part most worth stealing. You can't just record an agent flailing and label the lucky runs. So the paper splits the job between two roles.

An **answer-aware Tutor** knows the gold answer and builds the evidence chain *backward*, one hop at a time — from the final answer back toward the question. Working backward is what makes multi-hop tractable: at each step the Tutor proposes a shell command, runs it, and verifies the retrieved document actually supports the current target before extracting the bridge entity for the next hop. There's a strict anti-cheating rule: commands are *target-masked*, forbidden from grepping for the answer entity or its aliases, so the trajectory reflects realistic information-seeking rather than "search for the thing I already know."

Then an **answer-blind Planner** rebuilds that verified chain *forward*, drafting reasoning and actions using only what an agent could actually see at that point in time. The Tutor does a constrained edit pass to make the Planner's reasoning justify each verified command without leaking future state. The result: trajectories that are causally honest (forward-looking, no peeking) but evidence-reliable (backward-verified). Everything gets filtered — the final answer must overlap the gold answer, and a judge discards any trajectory whose reasoning quietly references facts not yet observed.

That cold-start set (10k trajectories) seeds an SFT pass that installs the structural habits — `rg -F` for exact matching, `| head` to cap output, cascaded `rg ... | rg ...` filtering. Only *then* does GRPO take over, sampling five trajectories per question and rewarding token-F1 on the final answer, gated by a format check so malformed runs earn nothing. The ablations are blunt about how load-bearing each stage is: drop GRPO and multi-hop performance collapses; drop SFT and training is so unstable they could only report a pre-collapse checkpoint.

A nice detail from the training dynamics: GRPO doesn't teach the agent to search *more*. The SFT-installed primitives (pipe depth, fixed-string matching, truncation) stay basically frozen. What RL changes is higher-level strategy — the agent learns to issue *fewer* commands by composing richer multi-stage pipelines, while spending more tokens reasoning between them.

Why This Matters Beyond One Benchmark

The deeper lesson is about *where* you put the intelligence in a search stack. The dominant paradigm trains the agent to write better queries for a *fixed* retriever — the retriever stays a black box. GrepSeek collapses that boundary. There's no semantic bottleneck deciding in advance what's relevant; the agent controls matching, filtering, and composition at arbitrary granularity. That's why it shines on exact entity names, rare symbolic strings like chemical formulas, and bridge entities — the cases where embedding-level smoothing quietly merges a subsidiary with its parent company.

It also makes search *interpretable*. Every retrieval step is a literal shell command you can read, replay, and debug. That's a meaningful operational property when your agent is wrong and you need to know why.

The honest caveat, which the paper doesn't hide: pure lexical search is brittle to surface-form variation. A missing diacritic or an unexpected accent can make the agent whiff entirely, and `rg` has no notion of semantic ranking, so an authoritative passage can get buried behind earlier file-order matches. On long-tail-entity datasets like PopQA, dense retrievers still win. The authors frame DCI as a complement to index-based retrieval, not a wholesale replacement, and point to hybrid architectures as future work.

What You Could Try on Monday

Three concrete moves. First, on any corpus small enough to grep — a docs set, a codebase, an internal wiki — try handing your agent `rg` over the raw files *instead of* standing up a vector DB. You skip the embedding pipeline and the index entirely, and for exact-match-heavy questions you may come out ahead. Second, steal the Tutor/Planner pattern whenever you need verified agentic trajectories: build the evidence chain backward with answer access and target-masking, then replay it forward answer-blind to keep it causally honest. It's a general recipe for synthesizing process-supervised data, not just for search. Third, mind the sequencing — SFT the search *primitives* first, then run GRPO to refine *strategy*. The order is not optional.

GrepSeek trains the agent to *gather* evidence by searching the corpus directly. But gathering is only half the job: once that evidence — often long, noisy, and full of near-miss distractors — lands in the context window, the agent still has to reason over it correctly. That's exactly where the next chapter picks up. LongTraceRL learns long-context reasoning from search-agent trajectories, using gold entities along the reasoning chain as entity-level rubric rewards for fine-grained process supervision, with distractors mined from real search runs. Both papers lean on search-agent trajectories as training signal — GrepSeek to learn the searching, LongTraceRL to learn the thinking that has to follow it.

Chapter 5: Hard Distractors and the Reasoning You Can’t See

Paper: LongTraceRL: Learning Long-Context Reasoning from Search Agent Trajectories with Rubric Rewards · arXiv 2605.31584 · **Code:** github.com/THU-KEG/LongTraceRL

Here’s a failure mode you’ve probably seen if you’ve ever shipped a long-context reasoning model: it gets the right answer for the wrong reasons. You ask a multi-hop question, the model burns through a hundred thousand tokens of context, and it returns “Moroccan-Swedish” — which happens to be correct — while citing the wrong song at one of the intermediate hops. The binary reward says: perfect, full marks. The model learned nothing about reasoning. It learned that guessing sometimes pays.

LongTraceRL, out of Tsinghua, is a clean attack on exactly that gap. Its argument is that the two standard ingredients of long-context reinforcement learning — the training data and the reward signal — are both too easy on the model, and that fixing both with surprisingly cheap tricks buys real gains. On a 4B model it lifts the average across five benchmarks by 5.7 points over base and 2.5 points over the strongest baseline, with the biggest jumps on the hardest, most reasoning-heavy benchmark. The headline isn’t the score, though. It’s *how* they made the training harder, because the recipe is something you can borrow.

Mining distractors from agents that actually searched

Most long-context training data is built by taking a multi-hop question and padding the context with **distractors** — junk documents the model has to ignore. The dirty secret is that those distractors are usually random. A question about Lady Gaga’s producer gets padded with a blog post about Byzantine officials and a meteorologist’s diary. Off-topic noise is trivial to filter; the model learns to skim, not to reason. In their numbers, random distractors share a relevant entity with the reasoning chain just 1.35% of the time. There’s almost nothing to confuse the model.

LongTraceRL’s move is to let a real search agent answer each question first, then steal its trajectory. They generate eight-hop questions by random-walking the Wikipedia hyperlink graph, then deploy a deep-search agent that issues queries, opens documents, and cites sources. Every question gets answered five times; only trajectories that land the correct answer are kept. Then they harvest the wreckage. Documents the agent **opened but didn’t cite** become Tier-1 distractors — high confusability, because the agent itself thought they were worth reading. Documents that showed up in search results but were never opened become Tier-2 — low confusability, only surface-related. Assembly fills the context with Tier-1 first, then Tier-2, then shuffles so position leaks nothing.

The payoff is measurable. Tier-1 distractors share a rubric entity with the reasoning chain 63% of the time, versus 15% for naive one-shot search and that pathetic 1.35% for random. Harder distractors, in their ablation, track downstream score almost monotonically. The model is finally being forced to *discriminate* rather than skim.

Rewarding the reasoning, not just the answer

The second fix targets the reward. Because the question-generation step emits the gold entities along each reasoning chain, they get free process supervision: a **rubric reward** equal to the fraction of gold entities the model actually surfaces in its reasoning. Hit four of five entities, score 0.8. That's entity-level supervision — finer than the chunk-level or document-level rewards other long-context RL papers use, and it doesn't need an auxiliary scoring model the way co-evolving reward schemes do.

The trap with any recall-based reward is obvious: the model can game it by enumerating every entity-looking string in the context without reasoning at all. The defense here is a **positive-only strategy** — the rubric reward is granted only to responses that already got the final answer right. Wrong answers get zero, full stop. So the rubric never rewards entity-stuffing on its own; it only differentiates *among* correct answers, pushing the model toward the ones that earned it. Their ablation makes the case bluntly: turn off the positive-only gate and let every rollout collect rubric credit, and the average drops from 59.0 to 57.1, with the reasoning-heavy benchmarks bleeding nearly five points each. Without that gate, every wrong rollout still banks rubric reward, the gradient stops caring about correctness, and the policy drifts toward enumerating gold-like entities. Strip the rubric reward out entirely and the gain almost vanishes — it's the dominant driver, not the data alone.

Why this travels beyond one paper

The deeper lesson is about where supervision comes from when the answer is binary but the work is long. Two ideas here generalize past long-context QA into any multi-turn agent RL setting.

First: your agents are already generating high-quality negatives, and you're throwing them away. A trajectory where an agent opened a document and then declined to cite it is a labeled hard negative — relevant enough to read, wrong enough to discard. That's a better adversarial signal than embedding similarity, and it costs you nothing but logging. If you run search or tool-using agents in production, you're sitting on a distractor mine.

Second: outcome-only rewards in long-horizon tasks are noisy in a specific, correctable way. A correct final answer reached through a broken intermediate step is a false positive your reward can't see. If any structure in your task gives you intermediate checkpoints — entities that must appear, tools that must be called, facts that must be retrieved — you can convert that structure into dense process reward. The positive-only gate is the part to remember: process reward applied to wrong answers is an invitation to reward hacking; applied only to right answers, it's a tiebreaker that rewards genuine reasoning.

What a practitioner can borrow

If you’re tuning a multi-turn agent with RL, three concrete moves fall out of this. Start logging your agents’ full trajectories and split retrieved-but-unused documents into opened-not-cited versus never-opened — the first tier is your hard distractor supply, and it’s free. If your task generation emits intermediate gold facts, wire them into an entity-recall rubric reward as a cheap process signal you don’t need a reward model to compute. And whatever process reward you add, gate it on outcome correctness — let it discriminate among the answers that already passed, never substitute for passing. The tuning knob matters too: their rubric weight peaked at 0.3, with 0.5 actively hurting as it diluted the outcome objective. There’s a real ceiling on how hard you can lean on process reward before it stops helping.

What’s quietly satisfying about LongTraceRL is how little of it is exotic. No new architecture, no auxiliary verifier, no clever loss. It’s standard GRPO fed harder data and a gated tiebreaker reward, both manufactured from byproducts the pipeline was already producing. The gains came not from a bigger hammer but from refusing to let the model off easy — making the distractors genuinely confusing and making the reward genuinely care whether the reasoning held up. That’s a good note to end a reading on: the next jump in long-context reasoning may not be waiting on a new method so much as on us being honest about how easy our training has been.